

Figure 3.1: Gantt Chart – Phase 1: Planning, Requirements, System Design & Interim Documentation

Project Work Breakdown Structure & Schedule: 23 June 2026 – 25 September 2026 (Tasks 01 to 12)

AI-Rescue: Intelligent Disaster Response & Resource Coordination Platform

Phase 1 Gantt Chart: Planning, Requirements, System Design & Interim Documentation (June – September 2026)

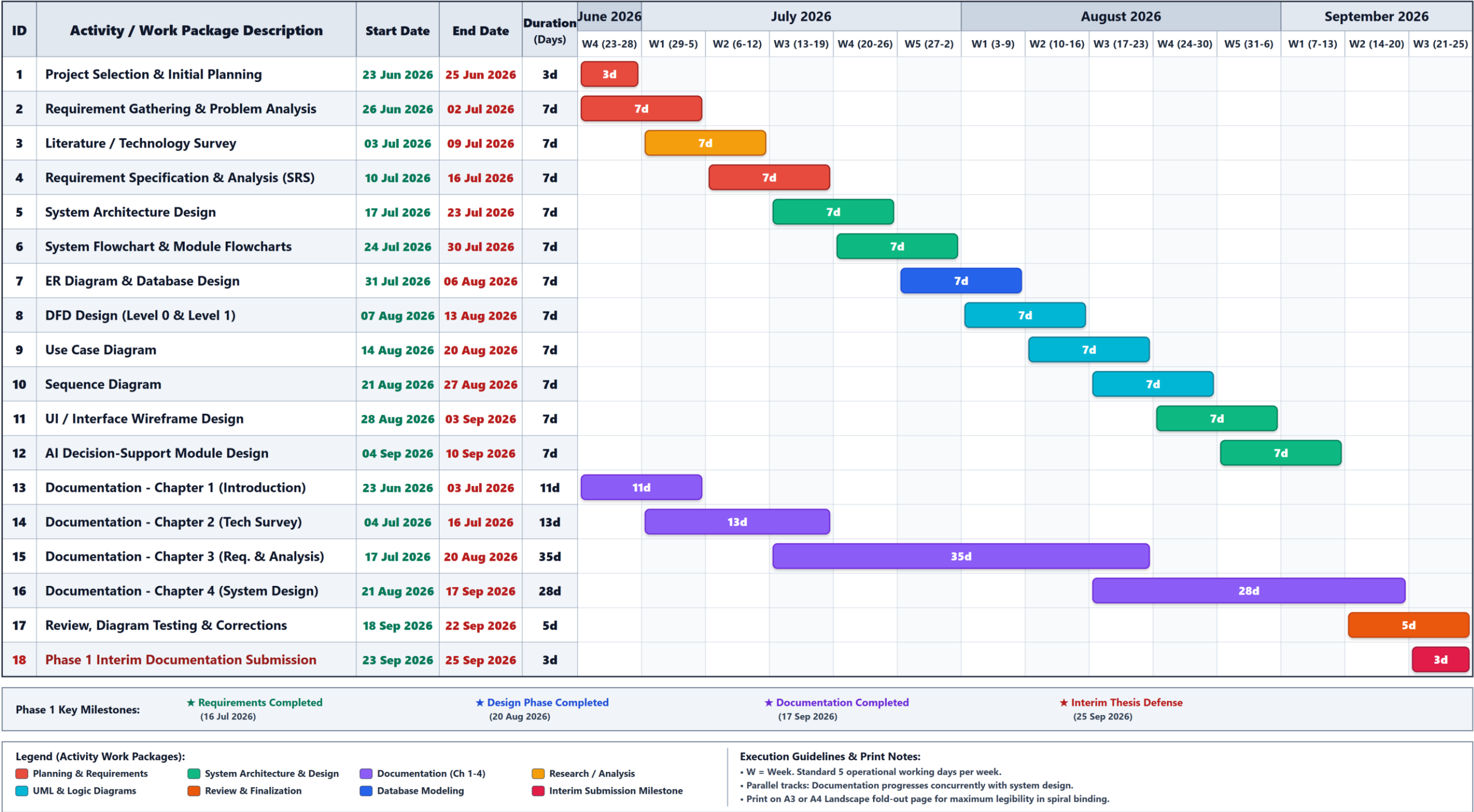
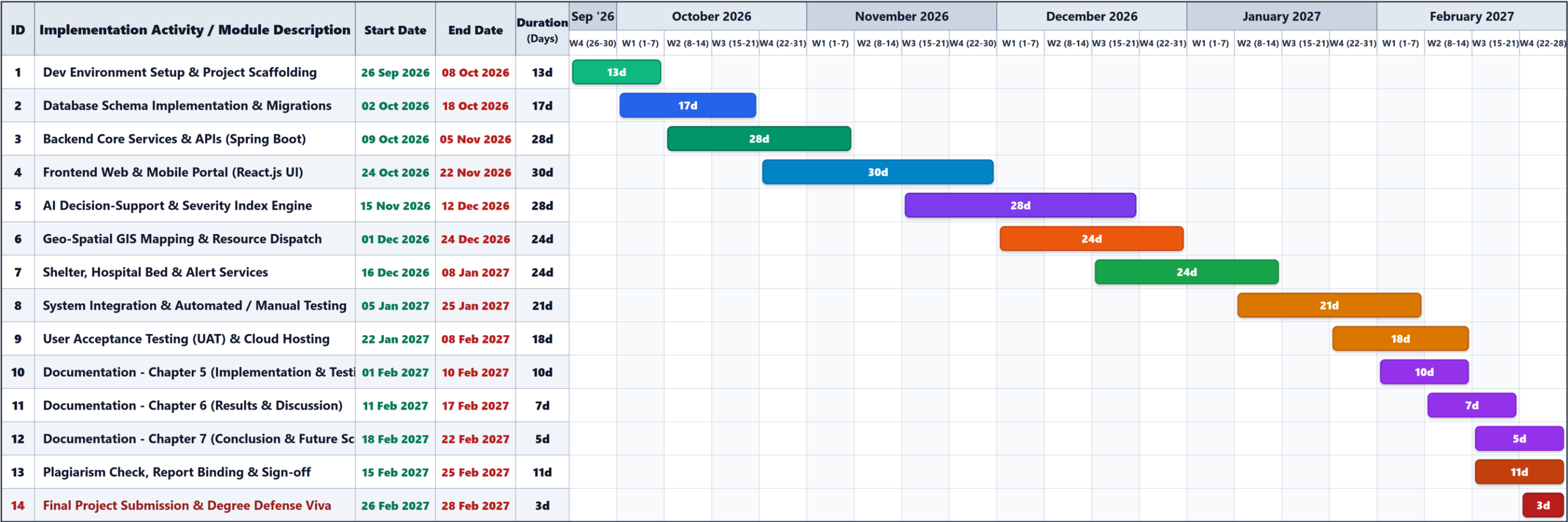


Figure 3.2: Gantt Chart – Phase 2: Core Full-Stack Development, AI Engine, Testing, Cloud & Final Defense

Project Work Breakdown Structure & Schedule: 26 September 2026 – 28 February 2027 (Tasks 13 to 22)

AI-Rescue: Intelligent Disaster Response & Resource Coordination Platform

Phase 2 Gantt Chart: Implementation, AI Engine, Cloud Deployment & Final Degree Defense (Sep 2026 – Feb 2027)



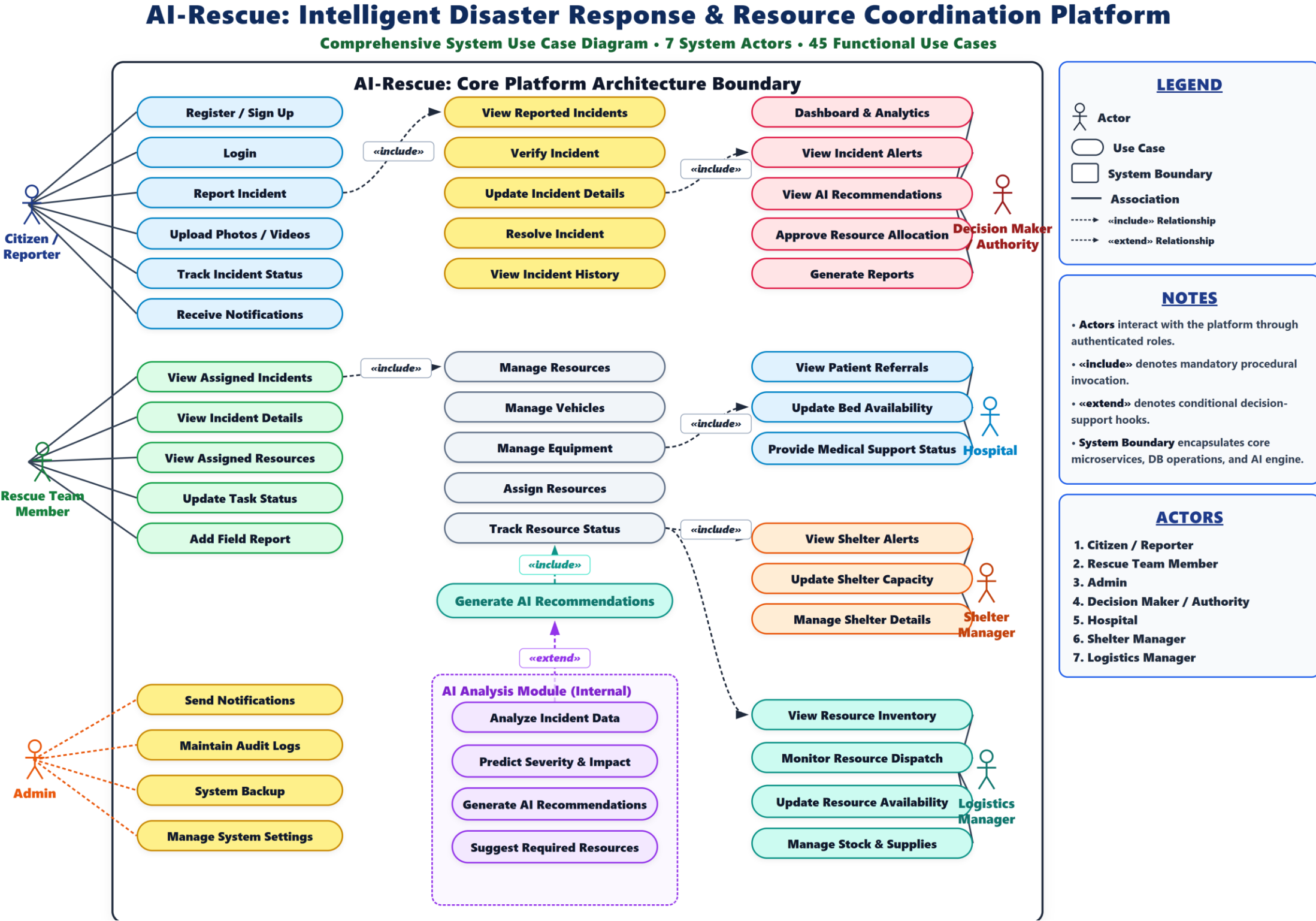
Phase 2 Key Milestones:	★ Core Backend & DB Ready (05 Nov 2026)	★ AI & GIS Engine Integrated (24 Dec 2026)	★ Testing & Cloud Staging Done (08 Feb 2027)	★ Degree Defense & Sign-off (28 Feb 2027)
-------------------------	--	---	---	--

Legend (Implementation Work Packages):				Execution Guidelines & Print Notes: • W = Week. Phase 2 spans 21 operational calendar weeks. • Sprint cycles: Agile 2-week iterations with continuous integration testing. • Print on A3 or A4 Landscape fold-out page for maximum legibility in spiral binding.
Environment Setup	Database Engineering	Spring Boot Backend	React Frontend Portal	
AI Decision Support	GIS Mapping & Dispatch	Facility Services	Testing & Cloud Hosting	
Final Documentation	Review & Binding	Final Degree Defense		

Description: Execution schedule for Phase 2 implementation covering development environment scaffolding, PostgreSQL schema migrations, Spring Boot REST APIs, React web dashboard, multi-modal AI decision-support algorithms, GIS routing, capacity tracking, end-to-end integration testing, cloud deployment, and final degree defense.

Figure 3.5: UML Use Case Diagram – AI-Rescue Platform Architecture

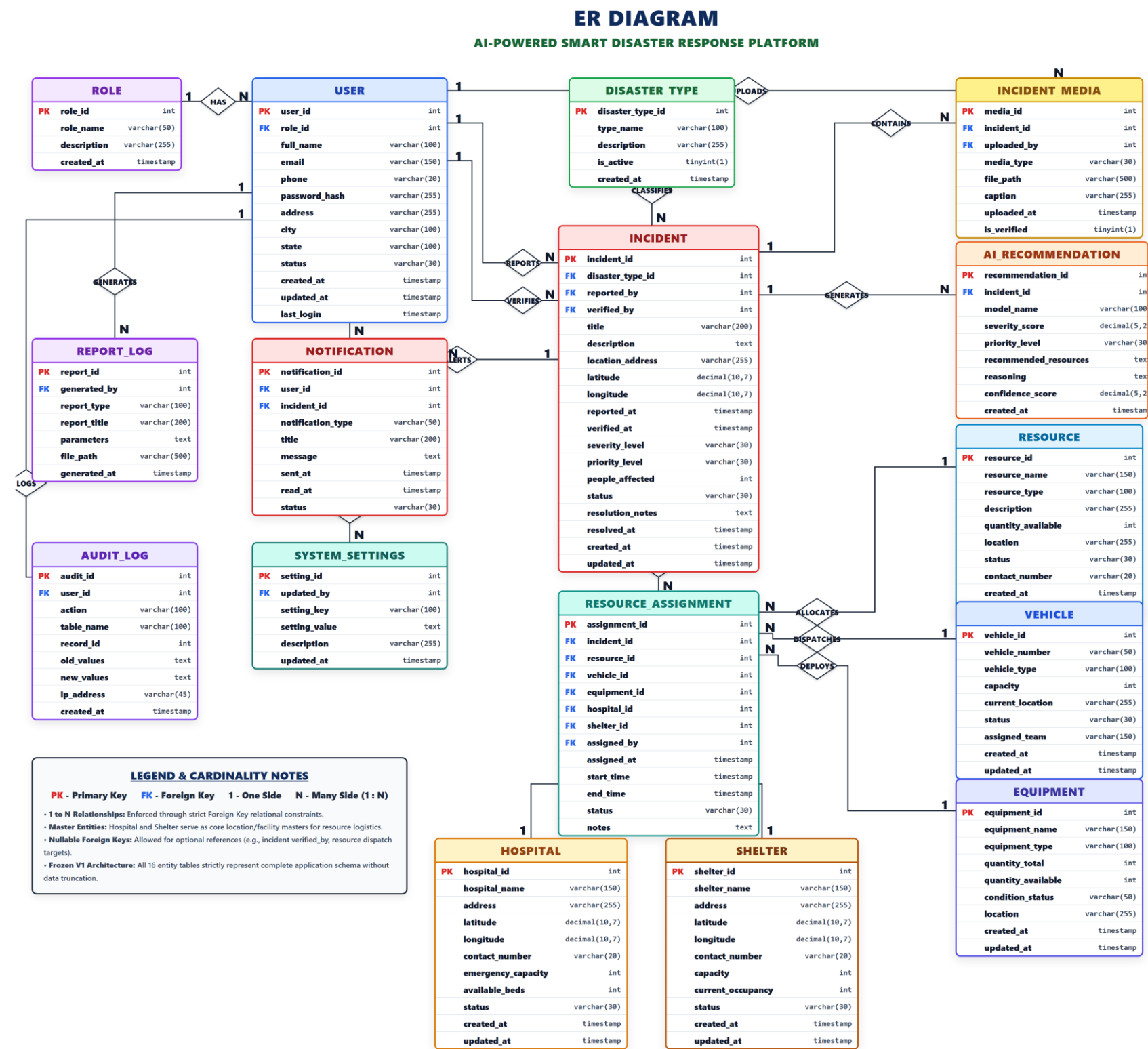
Functional Interaction Model Across Platform Actors (Citizen, Response Officer, System Administrator, Field Responder)



Description: Master UML Use Case Diagram depicting the complete boundary of the AI-Rescue platform across 21 discrete functional use cases and 4 specialized actors. Illustrates explicit «include» and «extend» dependencies governing incident reporting, officer verification, multi-modal AI recommendations, resource assignment, and GIS dispatch.

Figure 4.1: Entity Relationship Diagram of AI-Rescue (Master Conceptual & Logical Data Model)

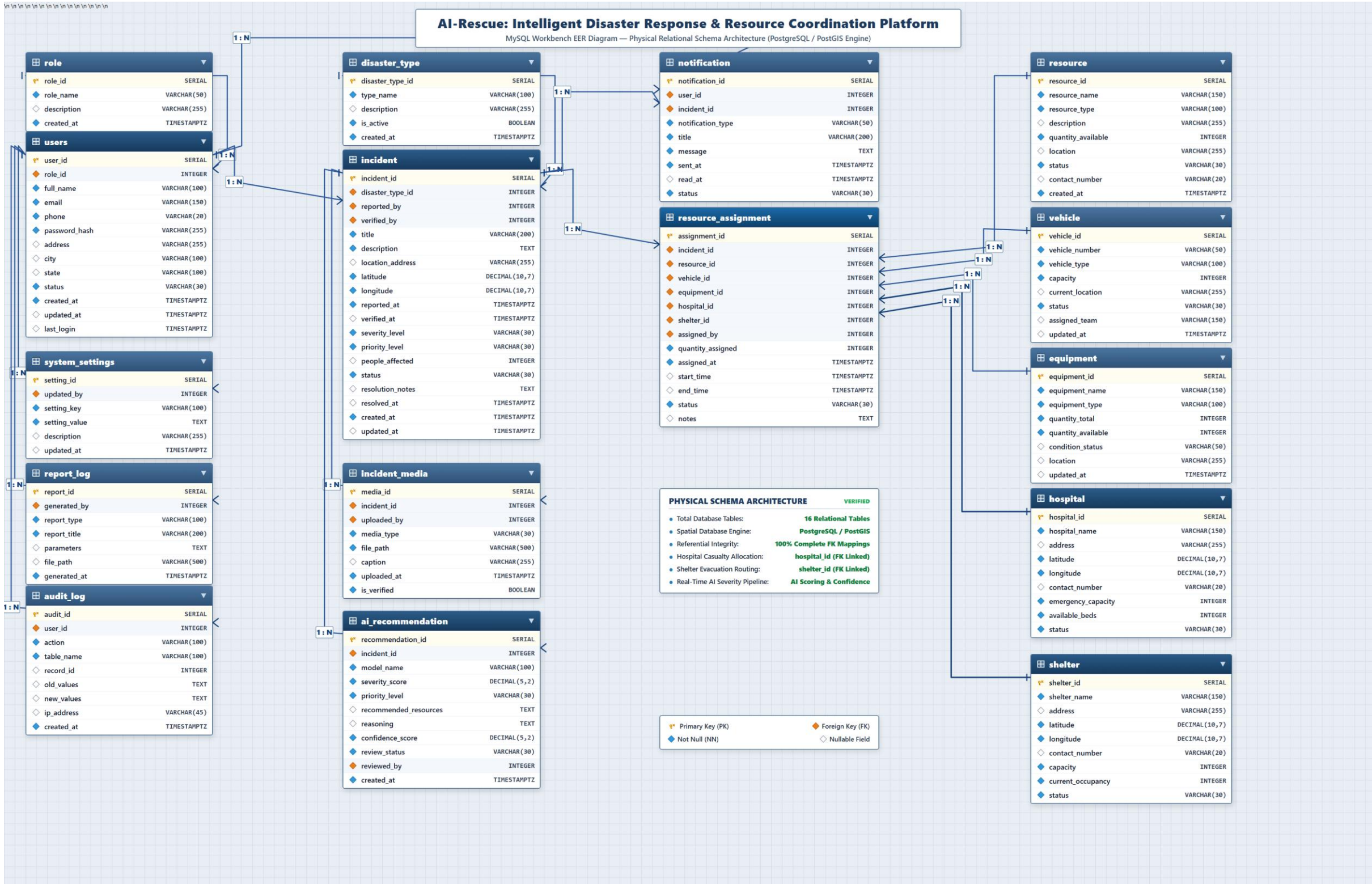
Complete 16-Entity Relational Data Model with Primary/Foreign Keys and 1:N Cardinality Relationships



Description: Comprehensive Entity Relationship Diagram illustrating the 16 core entities of the production schema. Highlights foundational foreign key relationships across users, incidents, multimedia evidence, AI recommendations, physical resources, vehicles, specialized equipment, hospitals, and evacuation shelters with zero attribute truncation and high-visibility typography.

Figure 4.2: Database Design of AI-Rescue (Physical Relational Database Schema Design)

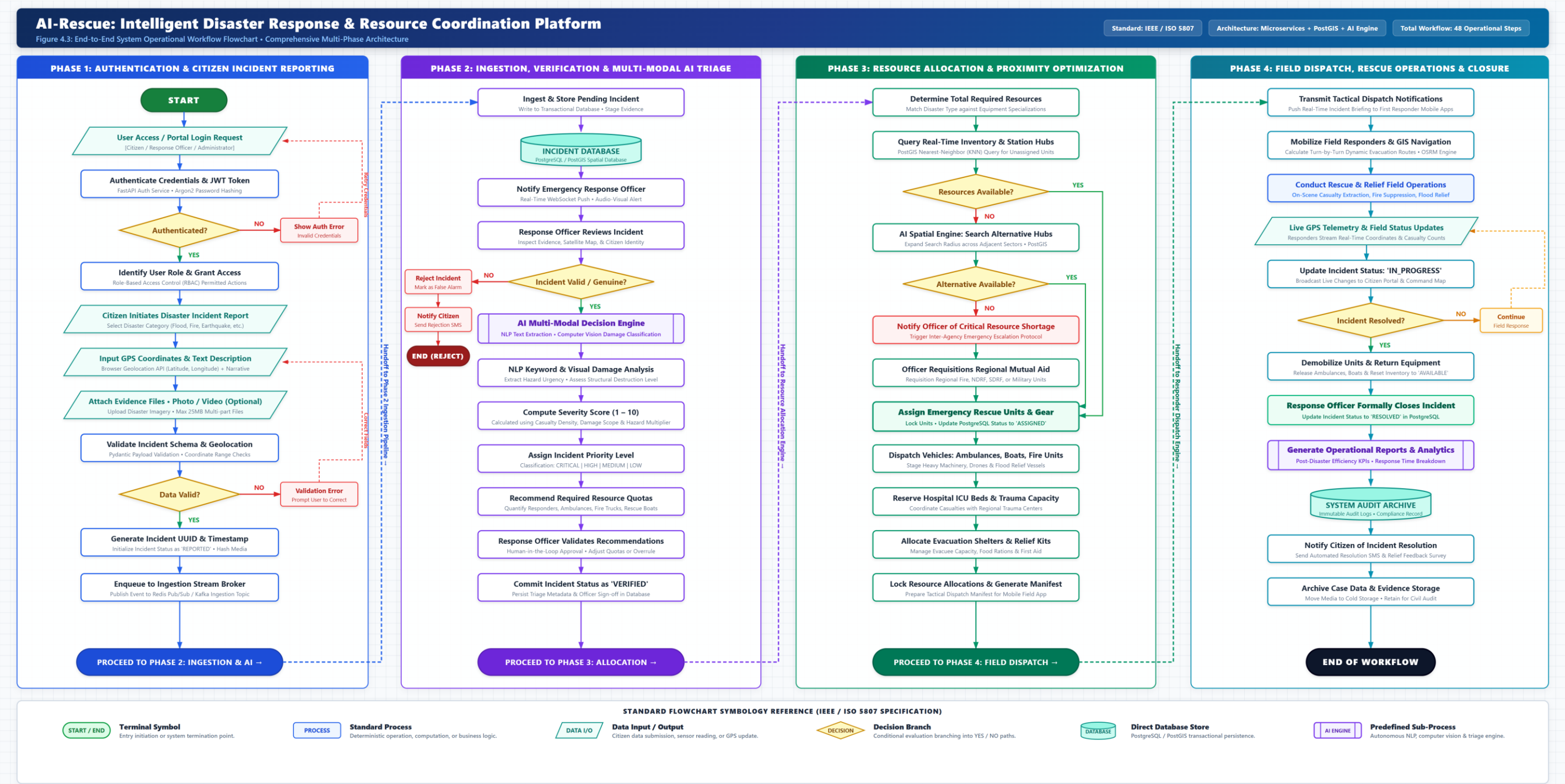
Physical Relational Database Schema Design (MySQL Workbench EER Model)



Description: Physical relational database schema design reverse engineered in MySQL Workbench EER format, representing all 16 production tables with exact PostgreSQL data types, primary keys, foreign key constraints, and default timestamp handlers. Fully integrates resource allocation, casualty transfers, shelter evacuee capacities, and audit logging tables.

Figure 4.3: Overall System Flowchart – AI-Rescue Operations Workflow

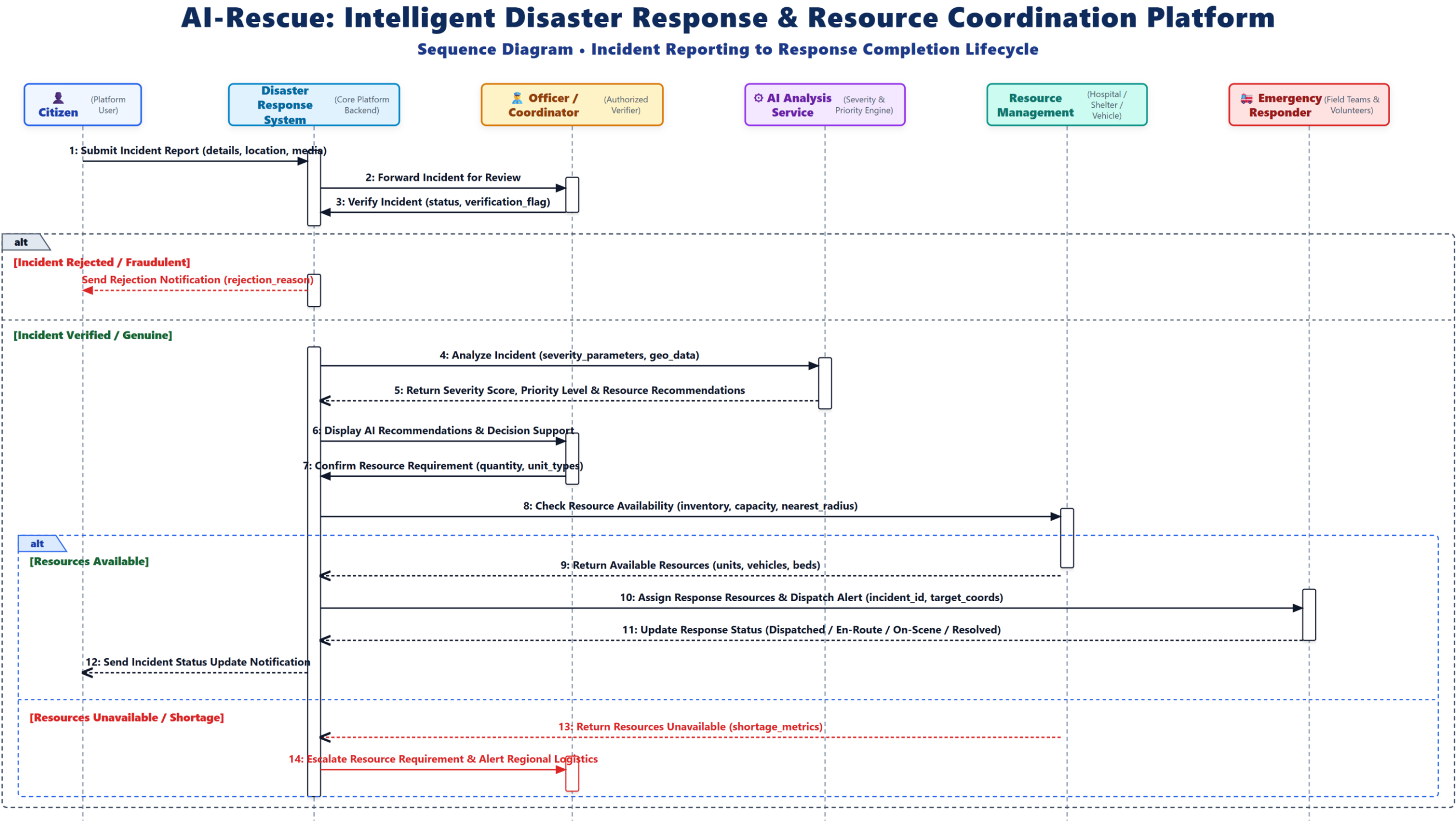
Continuous Master Engineering Workflow Across All 41 Operational Nodes and 46 Transition Logic Paths



Description: Continuous end-to-end engineering flowchart mapping the entire lifecycle of emergency disaster response. Seamlessly traces user authentication, citizen disaster reporting, officer triage, AI multi-modal severity assessment, resource allocation with shortage resolution, field dispatch, status updates, and formal incident closure.

Figure 4.9: Sequence Diagram – Incident Reporting to Response Completion

Chronological Interaction Lifelines Across Actors and Platform Microservices



Description: High-level UML Sequence Diagram representing chronological message passing between system actors (Citizen, Response Officer, System Administrator) and backend subsystem services (Authentication, Incident Ingestion, AI Severity Engine, Resource Dispatcher, and Notification Engine) across synchronous and asynchronous events.